

Freestanding Library: Character primitives and the C library

Document number: P2338R2

Date: 2021-11-11

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: SG22 (C Liaison), Library Evolution Working Group

Abstract

Add everything to the shared C and C++ freestanding library that can be implemented without OS calls and space overhead. Also add primitive character operations (<charconv> and char_traits) to the C++ freestanding library.

Change history

R2

- Rebase onto WG14's N2731 working draft
- Made <wchar.h> functions optional in freestanding C implementations
- Added strtok to C++ freestanding implementations, because C already did it
- Discussed C's Annex K
- Removing erroneous mention of <ratio> from [compliance] wording
- Finishing <cinttypes> removal in the C++ wording

R1

- Commenting on POSIX error handling
- Removing <cinttypes>, as WG14 is deprecating intmax_t
- Adding SDCC state of implementation
- Discussing design alternative of optional features
- Further discussing wchar_t functions

Introduction

The current definition of the freestanding implementation is not very useful. Here is the current high level definition from WG21's [intro.compliance]:

- 7 Two kinds of implementations are defined: a *hosted implementation* and a *freestanding implementation*. For a hosted implementation, this document defines the set of available libraries. A freestanding implementation is one in which execution may take place without the benefit of an operating system, and has an implementation-defined set of libraries that includes certain language-support libraries ([compliance]).

Similar wording is present in 5.1.2.1 "Freestanding Environment" in WG14 N2454.

In a freestanding environment (in which C program execution may take place without any benefit of an operating system)[...]

The main people served by the current C++ freestanding definition are people writing their own hosted C++ standard library to sit atop the compiler author's freestanding implementation (i.e. the STLport use case). The C++ freestanding portions contain most of the functions and types known to the compiler that can't easily be authored in a cross-compiler manner.

The current set of freestanding libraries provides too little to kernel, micro-controller, and GPU programmers. Why should a systems programmer need to rewrite std::from_chars or memcpy()?

I propose we provide the (nearly) maximal subset of the library that does not require an OS or space overhead. In order to continue supporting the "layered" C++ standard library users, we will continue to provide the (nearly) minimal subset of the library needed to support all the language features, even if these features have space overhead. Language features requiring space overhead or OS support will remain intact.

Motivation

The C and C++ standard libraries have many generally useful facilities that systems programmers could benefit from. By requiring those functions to be present in freestanding implementations, we make it possible to make higher level programs both easier to write, and portable. Currently, programs that would like to be portable are required to either rely on implementation defined extensions, or provide look-alike implementations.

Current State

The requirements on freestanding implementations have diverged over time between C and C++.

C

A freestanding C implementation is required to provide the entirety of the following headers:

- `<float.h>`
- `<iso646.h>`
- `<limits.h>`
- `<stdalign.h>`
- `<stdarg.h>`
- `<stdbool.h>`
- `<stddef.h>`
- `<stdint.h>`
- `<stdnoreturn.h>`

Most of `<string.h>` is required (`strdup`, `strndup`, `strcoll`, `strxfrm`, and `strerror` are excluded). This includes `strtok`, which requires global data.

Some additional features are required if the implementation defines the `__STDC_IEC_60559_BFP__` (binary floating point) macro or the `__STDC_IEC_60559_DFP__` (decimal floating point) macro. This includes `<fenv.h>`, `<math.h>`, and parts of `<stdlib.h>`. Such implementations indirectly require locale support, as the `<stdlib.h>` numeric conversion functions are implemented in terms of `isspace`.

The entire core language is required. This includes `_Thread_local`, which requires operating system interaction on multi-threaded systems.

C++

A freestanding C++ implementation is required to provide the entirety of the following headers:

- `<cstdint>`
- `<cfloat>`
- `<climits>`
- `<limits>`
- `<version>`
- `<cstdint>`
- `<new>`
- `<typeinfo>`
- `<source_location>`
- `<exception>`
- `<initializer_list>`
- `<compare>`
- `<coroutine>`
- `<cstdarg>`
- `<concepts>`

- `<type_traits>`
- `<bit>`

Almost all of `<atomic>` is required (C does not require `<stdatomic.h>` in freestanding implementations). `<cstdlib>` must provide `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit`.

The entire core language is required. For C++, this is much more onerous than for C, as the C++ core language includes exceptions, RTTI, thread-safe static initialization, and heap allocations.

The in-flight paper [P2013R4](#) makes it such that the allocating forms of `::operator new` are allowed to do nothing. This requirement often meant that the underlying C implementation of a freestanding C++ library needed to have `malloc` and `free` implementations.

The in-flight paper [P1642](#) adds many C++ specific facilities, but it also adds `_Exit`. The specification for `quick_exit` specifically calls out `_Exit`, so this omission is a specification bug.

A freestanding C++ implementation is mostly a superset of a freestanding C implementation, even in the "C" parts of C++. This means that a freestanding C++ implementation can not generally be built on top of a minimal freestanding C implementation. Either the C++ implementation must provide some of the C parts, or the C++ implementation will require a C implementation that provides more than the minimum.

Scope

The current scope of this proposal is limited to the freestanding standard library available to micro-controller, kernel, and GPU development.

This paper is currently concerned with the divisions of headers and library functions as they were in C++17. "[Standard Library Modules](#)" ([P0581](#)) discusses how the library will be split up in a post-modules world. This paper may influence the direction of P0581, but this paper won't make any modules recommendations.

Impact on the standards

In the C standard library, a new paragraph with bullets (and sub-bullets!) will be added that enumerates the full contents of the freestanding C library in prose. Prior to this paper, the required contents of the C freestanding library were spread over two paragraphs, with somewhat broken wording in the case of `<string.h>`.

In the C++ standard library, the editorial strategy described in WG21 [P1642](#) will be used to annotate which facilities are required in freestanding implementations.

Impact on implementations

C freestanding libraries would be required to provide more facilities than they are currently required to provide. Implementations likely already provide many of these functions due to user demand.

In theory, providing additional headers could silently break customer code that was already providing those headers. Those uses were undefined behavior according to WG14 N2454, 7.1.2 Standard Headers#4.

If a file with the same name as one of the above `<` and `>` delimited sequences, not provided as part of the implementation, is placed in any of the standard places that are searched for included source files, the behavior is undefined.

A C program could be using its own definition of, say, `memcpy`, so long as it does not include `string.h`. Implementations that are worried about such cases will need to take care to use macro definitions for most functions that forward to reserved identifier functions, so as to avoid multiple definitions.

C++ standard library headers will likely need to add preprocessor feature toggles to portions of headers that would emit warnings or errors in freestanding mode. The timeliness (compile time vs. link time) of errors remains a quality-of-implementation detail.

A minimal freestanding C17 standard library will not be sufficient to provide the C portions of the C++ standard library. `std::char_traits` and many of the function specializations in `<algorithm>` are implemented in terms of non-freestanding C functions. In practice, most C libraries are not minimal freestanding C17 libraries. The optimized versions of the `<cstring>` and `<wchar>` functions will often be the same for both hosted and freestanding environments. The main way in which a hosted implementation of (for example) `memcpy` could differ between hosted and freestanding is that some freestanding implementations (e.g. kernel implementations) would not want `memcpy` to use vector / floating point registers.

My expectation is that no new C++ freestanding library will be authored as a result of this paper. Instead hosted libraries will be stripped down through some feature toggle mechanism to become freestanding.

Design decisions

Even more so than for a hosted implementation; kernel, micro-controller, and GPU programmers do not want to pay for what they don't use. As a consequence, I am not adding features that require global data storage, even if that storage is immutable.

Note that the following concerns are not revolving around execution time performance. These are generally concerns about space overhead and correctness.

This proposal doesn't remove problematic features from the language, but it does make it so that the bulk of the freestanding standard library doesn't require those features. Users that disable the problematic features (as is existing practice) will still have portable portions of the standard library at their disposal.

Note that we cannot just take the list of C++ `constexpr` functions and make those functions the freestanding subset. We also can't do the reverse, and make everything freestanding `constexpr` or conditionally `noexcept`. `memcpy` cannot currently be made `constexpr` because it must convert from `cv void*` to `unsigned char[]`. Several floating point functions could be made `constexpr`, but would not be permitted in freestanding. `constexpr` also allows allocations, which freestanding avoids.

We also cannot just take the list of everything that is conditionally `noexcept` and make those functions freestanding. The "Lakos Rule"[Meredith11] prohibits most standard library functions from being conditionally `noexcept`, unless they have a wide contract.

Regardless, if a function or class is `constexpr` or `noexcept`, and it doesn't involve floating point, then that function or class is a strong candidate to be put into freestanding mode.

In the future, it may make sense to allow all `constexpr` functions into freestanding, so long as they are used in a `constexpr` context and not invoked at runtime.

Optional C `<wchar.h>`, required C++ `<wchar>`

In C++, there are many `wchar_t` function overloads and specializations of templates, and many of them rely on the functions in `<wchar>`. In addition, the C++ specification generally avoids making features optional.

The C committee pushed back on requiring `<wchar.h>` on freestanding implementations. The main reasons cited were lack of utility, and unclear semantics of `wchar_t` in the wild.

This paper makes freestanding C implementations define the `__STDC_WCHAR_H_FREESTANDING_LIBRARY__` macro to a non-zero value if `<wchar.h>` meets the listed requirements. `__STDC_WCHAR_H_FREESTANDING_LIBRARY__` will be defined to 0 otherwise. This allows users to differentiate between old implementations that *might* otherwise be conforming, and new implementations that are definitely conforming (or not).

In the long term, C++ implementations will be able to detect whether the C implementation provides the necessary functions to implement `wchar_t` overloads and specializations via the `__STDC_WCHAR_H_FREESTANDING_LIBRARY__` feature test macro.

Alternative: Make the additions optional features in freestanding

Rather than the proposed approach, we could instead have all the new features be optional features in freestanding. A feature test macro could advertise the presence or absence of these features.

This approach is unlikely to succeed in C++. C++ has two major kinds of implementations (freestanding and hosted), and very few optional features. C++ has struggled to maintain a coherent freestanding implementation, and adding additional build modes is more likely to make things worse, rather than better.

On the other hand, C uses optional features much more frequently. A `__STDC_MINIMAL_FREESTANDING_LIBRARY__` macro advertising the feature is more likely to have success in the C working group. Still, if there are no objections to adding the new features directly to freestanding, then that will reduce the number of dialects in the wild. The optional feature approach in C is viable, but it is only an alternative for the case that the direct freestanding approach cannot gain consensus.

Also note that freestanding C++ will generally depend on this more featureful freestanding C, whether it is part of the core freestanding requirements, or guarded by a feature test macro.

Alternative: No `wchar_t`

libc++ has recently implemented a `_LIBCPP_HAS_NO_WIDE_CHARACTERS` build-time feature toggle that removes all utilities and specializations involving `wchar_t` from the library. This will likely satisfy most non-Windows freestanding users, but it does not align well with the idea of freestanding being the maximum subset of hosted C++ that doesn't require globals or system calls.

Split overload sets

In C++, `to_chars`, `from_chars`, and `abs` are overloaded on floating point and integral types. This paper is making the integral overloads required in freestanding implementations.

It would be undesirable for the behavior of a library or program to silently change when porting it from a freestanding implementation to a hosted implementation though. That could easily happen with this overload set if a user called `abs(0.5)`. If the floating point overloads were merely omitted, then `abs(0.5)` would call one of the integral overloads on a freestanding implementation.

To avoid this trap, the floating point overloads will be marked as *//freestanding delete*. Freestanding implementations can either `=delete` the function, or provide an implementation of the function that meets the hosted requirements. This will cause accidental uses of these functions to fail to compile, as `=delete` functions participate in overload resolution.

Note that split overload set problems already exist in the C++ standard. A translation unit that includes `<cinttypes>` and calls `abs(0.5)` may end up resolving the overload to `abs(intmax_t)`.

Exceptions

Exceptions either require external jump tables or extra bookkeeping instructions. This consumes program storage space.

In the Itanium ABI, throwing an exception requires a heap allocation. In the Microsoft ABI, re-throwing an exception will consume surprisingly large amounts of stack space (2,100 bytes for a re-throw in 32-bit environments, 9,700 bytes in a 64-bit environment). Program storage space, heap space, and stack space are typically scarce resources in micro-controller development.

In environments with threads, exception handling requires the use of thread-local storage.

RTTI

RTTI requires extra data in vtables and extra classes that are difficult to optimize away, consuming program storage space.

Thread-local storage

Thread-local storage requires extra code in the operating system for support. In addition, if one thread uses thread-local storage, that cost is imposed on other threads. Note that there are common environments (e.g. the kernels of all major desktop operating systems) that support multiple threads, but do not support arbitrary thread local variables.

The heap

The heap is a big set of global state. In addition, C++ heap exhaustion is typically expressed via exception. Some micro-controller systems don't have a heap. In kernel environments, there is typically a heap, but there isn't a reasonable choice of *which* heap to use as the default. In the Windows kernel, the two best candidates for a default heap are the paged pool (plentiful available memory, but unsafe to use in many contexts), and the non-paged pool (safe to use, but limited capacity). The C++ implementation in the Windows kernel forces users to implement their own global operator `new` to make this decision.

[P2013R4](#) allows freestanding C++ implementations to provide an empty global allocating `::operator new` implementations by default.

Floating point

Many micro-controller systems don't have floating point hardware. Software emulated floating point can drag in large runtimes that are difficult to optimize away.

Most operating systems speed up system calls by not saving and restoring floating point state. That means that kernel uses of floating point operations require extra care to avoid corrupting user state.

In C, the dynamic floating-point environment has thread storage duration. This drags in the same set of problems that thread-local storage has.

Functions requiring global or thread-local storage

These functions are (mostly) not being added to the freestanding library. Examples are the locale aware functions, the C random number functions, and functions relying on `errno`. POSIX does not require the use of `errno` in any of the functions proposed for addition.

`strtok` is being added to C++ because C has already added it. In addition, the quantity of global state here is small, and should be straightforward to optimize away when `strtok` is not used.

All of C's Annex K (the bounds-checking functions) rely on global storage for the runtime-constraint handler.

Experience

The musl, newlib, and uclibc-ng C libraries are all marketed towards embedded use cases, and are all frequently used in embedded environments. All of the C facilities that this paper adds to the freestanding requirements are already present in musl, newlib, and uclibc-ng. This includes `memcpy` and the `<wchar.h>` facilities.

SDCC includes all of the proposed `<string.h>` functions. It includes `bsearch`, `qsort`, `abs`, and `labs` from `<stdlib.h>`. SDCC also includes a few functions and types from `<wchar.h>` (`wcscmp`, `wcslon`, `mbstate_t`, and `wint_t`).

SDCC omits the various `div` function and `div_t` types. `llabs` is not currently implemented. The remainder of `<wchar.h>` is not provided.

The Linux kernel uses a custom C library, though that library is more minimal, and in non-standard locations. The Linux kernel has implementations of `bsearch` (in `<linux/bsearch.h>`) and all of the `<string.h>` functions except for `memcpy`, though the `<string.h>` functions are in `<linux/string.h>`. The `<wchar.h>` functions and most of the `<stdlib.h>` functions were not present.

The Microsoft Windows kernel also has a C implementation that is distinct from the one that ships from Microsoft Visual Studio. That C implementation contains all of the new freestanding requirements with the exception of `llabs` and `lldiv`.

On the C++ front, I have successfully tested Visual Studio's `char_traits` implementation with a C++14 era set of `libc++` tests, all in the Windows kernel. The integral `<charconv>` functions have not been tested, but I do not foresee any issues there.

Technical Specifications

Partial headers newly required for freestanding implementations

Portions of `<cstdlib>`

- `size_t`
- `div_t`
- `ldiv_t`
- `lldiv_t`
- `NULL`
- `bsearch`
- `qsort`
- `abs(int)`
- `abs(long int)`

- `abs(long long int)`
- `labs`
- `llabs`
- `div`
- `ldiv`
- `lldiv`

All the error #defines in `<cerrno>`, but not `errno`.

The `errc` enum from `<system_error>`.

Portions of `<charconv>`.

- `to_chars_result`
- `from_chars_result`
- `to_chars(integral)`
- `from_chars(integral)`

The `char_traits` class from `<string>`.

Portions of `<cstring>`.

- `memcpy`
- `memmove`
- `strcpy`
- `strncpy`
- `strcat`
- `strncat`
- `memcmp`
- `strcmp`
- `strncmp`
- `memchr`
- `strchr`
- `strcspn`
- `strpbrk`
- `strrchr`
- `strspn`
- `strstr`
- `memset`
- `strlen`

On C, include `memccpy` in `<string.h>`, in addition to what is mentioned above for `<cstring>`.

Portions of `<wchar>`. For C, these functions are only provided if `__STDC_WCHAR_H_FREESTANDING_LIBRARY__` is defined to 1 by the implementation (i.e. these functions are optional in freestanding C).

- `wscpy`
- `wcncpy`
- `wmemcpy`
- `wmemmove`
- `wscat`
- `wcsncat`
- `wscmp`
- `wcsncmp`
- `wmemcmp`
- `wcschr`
- `wcscspn`
- `wcspbrk`
- `wcsrchr`
- `wcsspn`
- `wcsstr`
- `wcstok`
- `wmemchr`
- `wcslon`
- `wmemset`

A small portion of `<cmath>` will be present.

- `abs(int)`
- `abs(long int)`
- `abs(long long int)`

Notable omissions

`errno` is not included as it is global state. In addition, `errno` is best implemented as a thread-local variable.

`error_code`, `error_condition`, and `error_category` all have string in the interface.

Many string functions (`strtol` and family) rely on `errno`.

`rand` isn't required to use thread-local storage, but good implementations do. I don't want to encourage bad implementations. (Prior revisions also excluded `strtok` on this basis, but WG14 has since added `strtok` to C freestanding).

`assert` is not included as it requires a `stderr` stream.

`_Exit` is not included as I do not wish to add more termination functions. I hope to remove most of them in the future. Program termination requires involvement from the operating system / environment.

`<cctype>` and `<cwctype>` rely heavily on global locale data.

The `abs`, `div`, `imaxabs`, and `imaxdiv` overloads in `<inttypes>` aren't included, as WG14 is deprecating `intmax_t`. In addition, these functions are rarely used, and of low general utility.

Potential removals

Here are some things that I am currently requiring, but could be convinced to remove.

- `<wchar>`

The `<wchar>` functions are implementable for freestanding environments. The Microsoft and EFI ecosystems (EFI was the successor to BIOS and the predecessor to UEFI) use `wchar_t` extensively. `std::char_traits<wchar_t>` is usually implemented in terms of the `<wchar>` functions.

Most ecosystems don't use `wchar_t` much though. UTF8's success is reducing the need for `wchar_t`. This would be implementation burden with little customer demand. Some linking tools also have trouble discarding unused functions, and mitigating that problem would be further implementer burden with little payoff.

- `long long` versions of functions

Some existing implementations do not currently include the `long long` versions of functions, like `llabs` and `lldiv`. These are not critical to the proposal. They are fine in freestanding philosophically though. `long long` is permitted to be the same size as `long` in the C and C++ standards.

Potential additions

Here are some things that I am not currently requiring, but could be convinced to add.

- Floating point support

Perhaps we don't worry about library portability in all cases. Just because kernel modes can't easily use floating point doesn't mean that we should deny floating point to the micro-controller space. Do note that most of `<cmath>` has a dependency on `errno`.

- `errno` and string functions like `strtol`

While `errno` is global data, it isn't much global data. Thread safety is a concern for those platforms that have threading, but don't have thread-local storage. Environments that don't support arbitrary thread local data could special case `errno`.

- C `<stdatomic.h>`

C doesn't currently require `<stdatomic.h>` in freestanding implementations, but C++ requires `std::atomic`. I don't currently recommend adding `<stdatomic.h>` to freestanding C implementations, as that would also require dealing with non-lock-free atomics. If others feel strongly about unifying this aspect of C and C++ freestanding implementations, then the facilities could be added.

C++ Feature Test Macros

A freestanding C++ implementation that provides support for this paper shall define the following feature test macros:

Name	Header	Notes
__cpp_lib_freestanding_char_traits	<string>	
__cpp_lib_freestanding_charconv	<charconv>	
__cpp_lib_freestanding_cstdlib	<cstdlib> and <cmath>	The only freestanding parts of <cmath> are abs overloads that are also covered in <cstdlib>
__cpp_lib_freestanding_cstring	<cstring>	
__cpp_lib_freestanding_cwchar	<cwchar>	
__cpp_lib_freestanding_errc	<cerrno> and <system_error>	Covers errc and <cerrno> #defines

The above macros are useful for detecting the presence of various facilities. The user can provide a hand-rolled replacement on old or non-conforming implementations, while using the toolchain's facilities when available. These macros follow the policies proposed in [P2198: Freestanding Feature-Test Macros and Implementation-Defined Extensions](#).

C Wording

Wording is based off of WG14's N2731.

Change in 4. Conformance

Insert the following paragraph prior to paragraph 6:

The *freestanding library facilities* are as follows:

- The contents of the standard headers <float.h>, <iso646.h>, <limits.h>, <stdalign.h>, <stdarg.h>, <stdbool.h>, <stddef.h>, <stdint.h>, and <stdnoreturn.h>.
- The following facilities from the <errno.h> standard header: **EDOM, EILSEQ, and ERANGE.**
- The contents of the standard headers <fenv.h> and <math.h>, if the implementation defines **STDC_IEC_60559_BFP** or **STDC_IEC_60559_DFP**.
- The following facilities from the standard header <stdlib.h>:
 - **bsearch, qsort, abs, labs, llabs, div, div_t, ldiv, ldiv_t, lldiv, and lldiv_t**
 - The numeric conversion functions (7.22.1) if the implementation defines **STDC_IEC_60559_BFP** or **STDC_IEC_60559_DFP**.
- The contents of the <string.h> standard header, except the following functions: **strdup, strndup, strcoll, strxfrm, strerror.**
- The following facilities from the <wchar.h> standard header, if the implementation defines **STDC_WCHAR_H_FREESTANDING_LIBRARY** to a non-zero value: **mbstate_t, wint_t, WEOF, wcsncpy, wcsnncpy, wmemcpy, wmemmove, wscat, wcsncat, wscmp, wcsncmp, wmemcmp, wcschr, wcsncpy, wcsrchr, wcspn, wcsstr, wcstok, wmemchr, wcslen, and wmemset.**

Change paragraph 6 as follows:

The two forms of *conforming implementation* are hosted and freestanding. A *conforming hosted implementation* shall accept any strictly conforming program. A *conforming freestanding implementation* shall accept any strictly conforming program in which the use of the features specified in the library clause (Clause 7) is confined to the contents of the standard headers <float.h>, <iso646.h>, <limits.h>, <stdalign.h>, <stdarg.h>, <stdbool.h>, <stddef.h>, <stdint.h>, and <stdnoreturn.h>. **freestanding library facilities. The strictly conforming programs that shall be accepted by a conforming freestanding implementation may include any standard library header that contains freestanding library facilities.** Additionally, a *conforming freestanding implementation* shall accept any strictly conforming program in which the use of the features specified in the header <string.h>, except the following functions: **strdup, strndup, strcoll, strxfrm, strerror.** A conforming

implementation may have extensions (including additional library functions), provided they do not alter the behavior of any strictly conforming program. **All identifiers that are reserved when a standard header is included in a hosted implementation are reserved when it is included in a freestanding implementation.**

Delete paragraph 7.

The strictly conforming programs that shall be accepted by a conforming freestanding implementation that defines `__STDC_IEC_60559_BFP__` or `__STDC_IEC_60559_DEF__` may also use features in the contents of the standard headers `<fenv.h>` and `<math.h>` and the numeric conversion functions (7.22.1) of the standard header `<stdlib.h>`. All identifiers that are reserved when `<stdlib.h>` is included in a hosted implementation are reserved when it is included in a freestanding implementation.

Change in 6.10.8.3 Conditional feature macros

Add the following item to the list in paragraph 1:

`__STDC_WCHAR_H_FREESTANDING_LIBRARY` The integer constant `yyymml`, or the constant 0, intended to indicate support for freestanding functions in `<wchar.h>`.

C++ Wording

Wording is based off [WG21 N4878](#) from 2020-12-15. This paper also assumes that [P1642](#) and [P2198](#) have been accepted and applied.

Change in [conventions]

Add a new paragraph to [freestanding.membership] (added in [P1642](#)).

? On a freestanding implementation, a *freestanding deleted function* is a function that has either a deleted definition or a definition meeting the corresponding requirements in a hosted implementation. In the associated header synopsis for such freestanding deleted functions, the items are followed with a comment that includes *freestanding deleted*. [Example:

 double abs(double j); *// freestanding deleted*

-end example]

Change in [compliance]

Change [tab:headers.cpp.fs]:

	Subclause	Header(s)
[...]	[...]	[...]
?? [support.start.term]?? [cstdlib.syn]	Start and terminationC standard library	<cstdlib>
[...]	[...]	[...]
?? [errno]	Error numbers	<cerrno>
?? [syserr]	System error support	<system_error>
?? [charconv]	Primitive numeric conversions	<charconv>
?? [string.classes]	String classes	<string>
?? [c.strings]	Null-terminated sequence utilities	<cstring>, <cwchar>
?? [c.math]	Mathematical functions for floating-point types	<cmath>
[...]	[...]	[...]

Change in [cstdlib.syn]

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `div_t`
- `ldiv_t`
- `lldiv_t`
- `bsearch`
- `qsort`
- `abs(int)`
- `abs(long int)`
- `abs(long longint)`
- `labs`
- `llabs`
- `div`
- `ldiv`
- `lldiv`

Please append a *// freestanding deleted* comment to the following entities:

- `abs(float)`
- `abs(double)`
- `abs(long double)`

Change in [\[version.syn\]](#)

Please add the following feature test macros to [\[version.syn\]](#):

```
#define cpp_lib freestanding_char_traits new-val // freestanding, also in <string>
#define cpp_lib freestanding_charconv new-val // freestanding, also in <charconv>
#define cpp_lib freestanding_cstdlib new-val // freestanding, also in <cstdlib>, <cmath>
#define cpp_lib freestanding_cstring new-val // freestanding, also in <cstring>
#define cpp_lib freestanding_cwchar new-val // freestanding, also in <cwchar>
#define cpp_lib freestanding_errc new-val // freestanding, also in <cerrno>, <system_error>
```

Change in [\[cerrno.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `E2BIG`
- `EACCES`
- `EADDRINUSE`
- `EADDRNOTAVAIL`
- `EAFNOSUPPORT`
- `EAGAIN`
- `EALREADY`
- `EBADF`
- `EBADMSG`
- `EBUSY`
- `ECANCELED`
- `ECHILD`
- `ECONNABORTED`
- `ECONNREFUSED`
- `ECONNRESET`
- `EDEADLK`
- `EDESTADDRREQ`
- `EDOM`
- `EEXIST`
- `EFAULT`
- `EFBIG`

- EHOSTUNREACH
- EIDRM
- EILSEQ
- EINPROGRESS
- EINTR
- EINVAL
- EIO
- EISCONN
- EISDIR
- ELOOP
- EMFILE
- EMLINK
- EMSGSIZE
- ENAMETOOLONG
- ENETDOWN
- ENETRESET
- ENETUNREACH
- ENFILE
- ENOBUFS
- ENODATA
- ENODEV
- ENOENT
- ENOEXEC
- ENOLCK
- ENOLINK
- ENOMEM
- ENOMSG
- ENOPROTOPT
- ENOSPC
- ENOSR
- ENOSTR
- ENOSYS
- ENOTCONN
- ENOTDIR
- ENOTEMPTY
- ENOTRECOVERABLE
- ENOTSOCK
- ENOTSUP
- ENOTTY
- ENXIO
- EOPNOTSUPP
- EOVERFLOW
- EOWNERDEAD
- EPERM
- EPIPE
- EPROTO
- EPROTONOSUPPORT
- EPROTOTYPE
- ERANGE
- EROFS
- ESPIPE
- ESRCH
- ETIME
- ETIMEDOUT
- ETXTBSY
- EWOULDBLOCK
- EXDEV

Change in [\[system_error.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the `errc` entity.

Change in [\[charconv.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `to_chars_result`
- `from_chars_result`
- `to_chars_result to_chars(char* first, char* last, see below value, int base = 10);`
- `from_chars_result from_chars(const char* first, const char* last, see below& value, int base = 10);`

Please append a *// freestanding deleted* comment to the following entities:

- `to_chars(char* first, char* last, float value)`
- `to_chars(char* first, char* last, double value)`
- `to_chars(char* first, char* last, long double value)`
- `from_chars(const char *first, const char *last, float& value, chars_format fmt = chars_format::general)`
- `from_chars(const char *first, const char *last, double& value, chars_format fmt = chars_format::general)`
- `from_chars(const char *first, const char *last, long double& value, chars_format fmt = chars_format::general)`

Change in [\[string.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `template<class charT> struct char_traits`
- `template<> struct char_traits<char>`
- `template<> struct char_traits<char8_t>`
- `template<> struct char_traits<char16_t>`
- `template<> struct char_traits<char32_t>`
- `template<> struct char_traits<wchar_t>`

Change in [\[cstring.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `memcpy`
- `memmove`
- `strcpy`
- `strncpy`
- `strcat`
- `strncat`
- `memcmp`
- `strcmp`
- `strncmp`
- `memchr`
- `strchr`
- `strcspn`
- `strpbrk`
- `strrchr`
- `strspn`
- `strstr`
- `strtok`
- `memset`
- `strlen`
- `NULL`

The following entities should **NOT** have freestanding comments appended to them:

- `strcoll`

- strxfrm
- strerror

Change in [\[cwchar.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- size_t
- mbstate_t
- wint_t
- wcscpy
- wcsncpy
- wmemcpy
- wmemmove
- wcscat
- wcsncat
- wcscmp
- wcsncmp
- wmemcmp
- wcschr
- wcscspn
- wcsrchr
- wcssp
- wcsstr
- wcstok
- wmemchr
- wcslen
- wmemset
- NULL
- WCHAR_MAX
- WCHAR_MIN
- WEOF

The following entities should **NOT** have freestanding comments appended to them:

- struct tm
- fwprintf
- fwscanf
- swprintf
- swscanf
- vfwprintf
- vfwscanf
- vswprintf
- vswscanf
- vwprintf
- vwscanf
- wprintf
- wscanf
- fgetwc
- fgetws
- fputwc
- fputws
- fwide
- getwc
- getwchar
- putwc
- putwchar
- ungetwc
- wcstod
- wcstof
- wcstold

- `wcstol`
- `wcstoll`
- `wcstoul`
- `wcstoull`
- `wcscoll`
- `wcsxfrm`
- `wcsftime`
- `btowc`
- `wctob`
- `mbsinit`
- `mbrlen`
- `mbrtowc`
- `wcrtomb`
- `mbsrtowcs`
- `wcsrtombs`

Change in [\[cmath.syn\]](#)

Instructions to the editor:

Please append a *// freestanding* comment to the following entities:

- `abs(int)`
- `abs(long int)`
- `abs(long long int)`

Please append a *// freestanding deleted* comment to the following entities:

- `abs(float)`
- `abs(double)`
- `abs(long double)`

Acknowledgements

Thanks to Philipp Krause and Rajan Bhakta for their feedback on this paper.